

# Memoria de Aprendizaje por Refuerzo

## Aprendizaje en entornos complejos

Joaquín Ayala Filardi  
Máster en Inteligencia Artificial

Curso 2025/2026

## 1. Introducción

En esta parte de la práctica se va a trabajar el aprendizaje en entornos complejos. Más concretamente se centra en dos entornos **Gymnasium: RiverCourierEnv** y **ThermalGliderEnv**.

El objetivo experimental es comparar agentes de aprendizaje por refuerzo en problemas donde la decisión depende del estado y de la evolución del entorno. Para ello, se hace uso de una misma interfaz de agente, un mismo bucle de interacción y métricas comunes, de modo que la comparación no dependa de diferencias accesorias entre los notebooks.

## 2. Entornos y bucle común de agente

### 2.1. Interfaz de entrenamiento

El código separa el entorno, el agente y el protocolo experimental. Todos los agentes heredan de una clase común con cuatro métodos: `begin_episode`, `act`, `observe` y `end_episode`. El bucle `run_episode` reinicia el entorno con semilla, pide una acción al agente, ejecuta `env.step(action)` y registra retorno, longitud, éxito, incidentes o violaciones.

Listing 1: Estructura común de interacción agente-entorno.

```
observation, _ = env.reset(seed=seed)
agent.begin_episode(training=training)
action = agent.act(observation, training=training)
while True:
    next_observation, reward, terminated, truncated, info = env.step(action)
    next_action = agent.observe(observation, action, reward, next_observation, terminated, truncated)
    if terminated or truncated:
        break
    observation = next_observation
    action = next_action if next_action is not None else agent.act(observation)
agent.end_episode(training=training)
```

Las métricas guardadas son retorno medio, longitud media del episodio, tasa de éxito y tasa de riesgo. En **RiverCourierEnv** el riesgo se mide en incidentes en el vado, mientras que en **ThermalGliderEnv** mediante violaciones de las restricciones del vuelo.

### 2.2. RiverCourierEnv

**RiverCourierEnv** es un entorno discreto de reparto sobre una cuadrícula de  $7 \times 7$ . El agente empieza en  $(6, 0)$  y debe llegar a  $(0, 6)$ . La fila central representa el río: el vado  $(3, 1)$  permite un camino corto pero arriesgado, mientras que el puente  $(3, 6)$  ofrece una ruta más segura y larga. El estado codifica la posición, la fase de marea y el nivel de daño con 441 estados discretos. Las acciones posibles para el agente son *arriba*, *derecha*, *abajo*, *izquierda* y *esperar*.

El entorno penaliza pasos, esperas, movimientos inválidos e incidentes, mientras a la vez premia el progreso hacia la meta y añade una recompensa al completar el reparto. La fase de marea cambia en cada paso, por lo que la acción de esperar puede ser útil si el agente decide cruzar por el vado.

### 2.3. ThermalGliderEnv

**ThermalGliderEnv** usa observaciones continuas con cuatro componentes: posición, velocidad, temperatura y batería. Las acciones en este caso son *planear*, *impulso*, *acelerar* y *refrigerar*. El objetivo es alcanzar la zona de aterrizaje

manteniendo la velocidad, la temperatura y la batería dentro de límites seguros.

### 3. Métodos implementados

#### 3.1. Control tabular

En `RiverCourierEnv` se realiza la comparación de cuatro agentes tabulares:

- **Monte Carlo on-policy**: control first-visit con política  $\varepsilon$ -soft. Actualiza al terminar el episodio usando retornos observados.
- **Monte Carlo off-policy**: usa importancia ponderada para actualizar una política objetivo codiciosa a partir de una política de comportamiento exploratoria.
- **SARSA**: método de diferencias temporales on-policy. La actualización usa la acción realmente elegida en el siguiente estado:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a)).$$

- **Q-Learning**: método de diferencias temporales off-policy. La actualización usa la mejor acción estimada en el siguiente estado:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left( r + \gamma \max_b Q(s', b) - Q(s, a) \right).$$

#### 3.2. Control con aproximación

En `ThermalGliderEnv` se comparan tres enfoques:

- **Q-Learning discretizado**: referencia tabular con una discretización gruesa de  $2 \times 2 \times 2 \times 2$ , Es decir, se cuenta con 16 estados.
- **SARSA semi-gradiente**: aproximación lineal con tile coding, 8 mosaicos y pesos por acción.
- **Deep Q-Learning**: red neuronal con dos capas ocultas de 64 unidades, repetición de experiencias, red objetivo y pérdida `smooth_l1`.

El agente DQN almacena transiciones en un búfer, entrena por minibatches y actualiza una red objetivo cada cierto número de pasos. La acción durante entrenamiento se selecciona con una política  $\varepsilon$ -greedy decreciente.

### 4. Protocolo experimental

#### 4.1. Estudio tabular

El estudio tabular utiliza 2600 episodios de entrenamiento y 5 semillas: 2026, 2037, 2048, 2059 y 2070. La evaluación se realiza sin exploración durante 300 episodios por semilla. Se comparan dos condiciones:

- **Marea nominal**: `hazard_scale=1`.
- **Crecida +35 %**: `hazard_scale=1.35`.

Los agentes tabulares comparten  $\gamma = 0,98$ . SARSA y Q-Learning usa  $\alpha = 0,16$ . Las políticas de exploración reducen  $\varepsilon$  durante el entrenamiento.

#### 4.2. Estudio aproximado

El estudio con aproximación usa 1200 episodios de entrenamiento y 3 semillas: 2026, 2041 y 2056. La evaluación se realiza sin exploración durante 250 episodios por semilla. Se comparan dos condiciones:

- **Viento nominal**: `wind_std=0.006`.
- **Turbulencia x2.3**: `wind_std=0.014`.

El protocolo no se limita a medir recompensa. También registra la longitud de episodio, el éxito y el riesgo, pues un agente puede llegar rápido pero violar restricciones, o puede mantener seguridad a costa de trayectorias más largas.

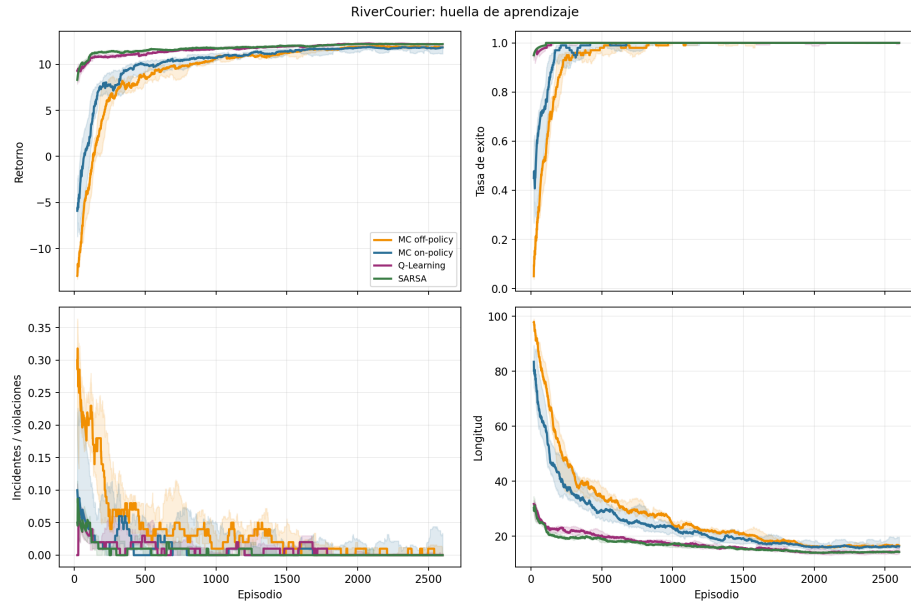


Figura 1: Huella de aprendizaje de los métodos tabulares.

## 5. Resultados del estudio tabular

| Agente        | Condición     | Retorno medio | Longitud media | Éxito  | Riesgo |
|---------------|---------------|---------------|----------------|--------|--------|
| MC off-policy | Crecida +35 % | 12.27         | 13.54          | 100 %  | 0 %    |
| SARSA         | Crecida +35 % | 12.23         | 13.79          | 100 %  | 0 %    |
| MC on-policy  | Crecida +35 % | 11.72         | 16.31          | 99.8 % | 2.3 %  |
| Q-Learning    | Crecida +35 % | 11.16         | 18.64          | 98 %   | 0.2 %  |
| MC off-policy | Marea nominal | 12.27         | 13.54          | 100 %  | 0 %    |
| SARSA         | Marea nominal | 12.23         | 13.79          | 100 %  | 0 %    |
| MC on-policy  | Marea nominal | 11.76         | 16.17          | 99.8 % | 1.9 %  |
| Q-Learning    | Marea nominal | 11.16         | 18.64          | 98 %   | 0.2 %  |

Tabla 1: Resultados agregados en RiverCourierEnv.

Monte Carlo off-policy y SARSA obtienen los resultados más sólidos: 100 % de éxito y riesgo nulo tanto en marea nominal como en crecida. En ambos casos la crecida no cambia el resultado agregado, lo que indica que las políticas aprendidas evitan depender del vado en situaciones peligrosas.

Monte Carlo on-policy mantiene una tasa de éxito muy alta, pero conserva una pequeña exposición al riesgo. Q-Learning también resuelve muchos episodios, aunque con menor retorno y mayor longitud media. La diferencia es relevante porque una longitud alta indica rutas menos directas o episodios con más correcciones.

La longitud del episodio se interpreta como indicador auxiliar. Cuando el retorno sube y la longitud baja, el agente está dejando de vagar y alcanza la meta de forma más directa. No obstante,  $f(t) = \text{len}(\text{episodio}_t)$  no basta por sí sola. Un fallo temprano también puede producir un episodio corto, por lo que se analiza junto con el éxito, el retorno y los incidentes.

| Agente        | Condición     | Éxito mínimo | Éxito máximo | Longitud media | Desv. longitud |
|---------------|---------------|--------------|--------------|----------------|----------------|
| MC off-policy | Crecida +35 % | 1            | 1            | 13.542         | 0.174          |
| MC off-policy | Marea nominal | 1            | 1            | 13.542         | 0.174          |
| MC on-policy  | Crecida +35 % | 0.99         | 1            | 16.307         | 2.972          |
| MC on-policy  | Marea nominal | 0.99         | 1            | 16.170         | 2.854          |
| Q-Learning    | Crecida +35 % | 0.9          | 1            | 18.641         | 11.151         |
| Q-Learning    | Marea nominal | 0.9          | 1            | 18.641         | 11.151         |
| SARSA         | Crecida +35 % | 1.00         | 1            | 13.793         | 0.446          |
| SARSA         | Marea nominal | 1.00         | 1            | 13.793         | 0.446          |

Tabla 2: Variabilidad entre semillas en el estudio tabular.

La variabilidad confirma que Q-Learning no termina con la misma política en todas las semillas. Su éxito mínimo baja a 0.90 y la desviación de longitud es mucho mayor que en el resto. Por ello, no se concluye superioridad solo a partir de una curva o de una diferencia pequeña en retorno medio.

Políticas aprendidas: atajo, espera y ruta segura

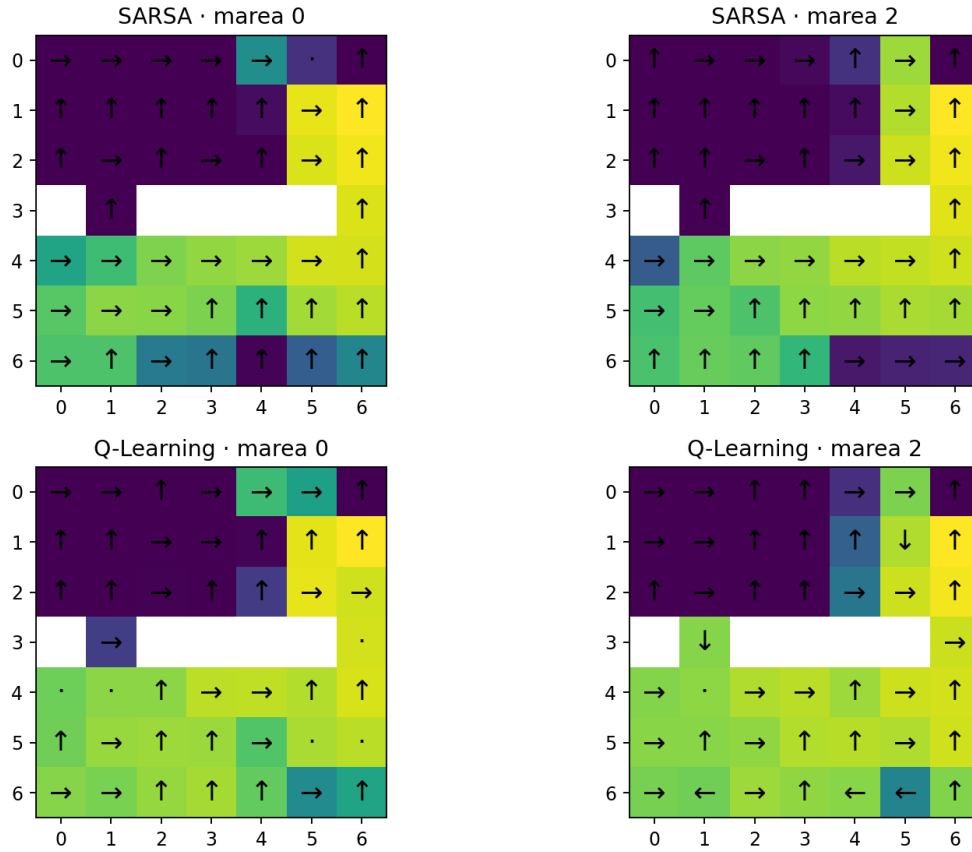


Figura 2: Mapas de política aprendida para inspección cualitativa.

Los mapas de política ayudan a inspeccionar si el agente usa el puente, espera o evita el vado. Aun así, las flechas en estados poco visitados no se consideran evidencia principal. La conclusión se apoya en las métricas agregadas y en la evaluación por semillas.

## 6. Resultados del estudio aproximado

| Agente                  | Condición        | Retorno medio | Longitud media | Éxito  | Riesgo |
|-------------------------|------------------|---------------|----------------|--------|--------|
| Deep Q-Learning         | Turbulencia x2.3 | 18.86         | 17.17          | 98 %   | 2.5 %  |
| Q-Learning discretizado | Turbulencia x2.3 | 18.44         | 19.13          | 96.9 % | 6.7 %  |
| SARSA semi-gradiente    | Turbulencia x2.3 | 18.10         | 12.28          | 94.5 % | 10 %   |
| Deep Q-Learning         | Viento nominal   | 19.32         | 16.67          | 99.6 % | 0.4 %  |
| SARSA semi-gradiente    | Viento nominal   | 19.41         | 11.70          | 99.5 % | 1.9 %  |
| Q-Learning discretizado | Viento nominal   | 19.18         | 19.52          | 99.7 % | 2.3 %  |

Tabla 3: Resultados agregados en **ThermalGliderEnv**.

En viento nominal, SARSA semi-gradiente obtiene el mayor retorno medio y los episodios más cortos. Sin embargo, al aumentar la turbulencia su tasa de riesgo sube al 10 % y el éxito baja al 94.5 %. Esto muestra que la política aprendida es eficiente, pero más agresiva.

DQN no es el método más corto pero sí el más robusto, pues bajo turbulencia mantiene 98 % de éxito y 2.5 % de riesgo. La referencia discretizada funciona en muchos episodios, aunque es más lenta y su riesgo sube al 6.7 % con turbulencia, coherente con una discretización demasiado gruesa del estado continuo.

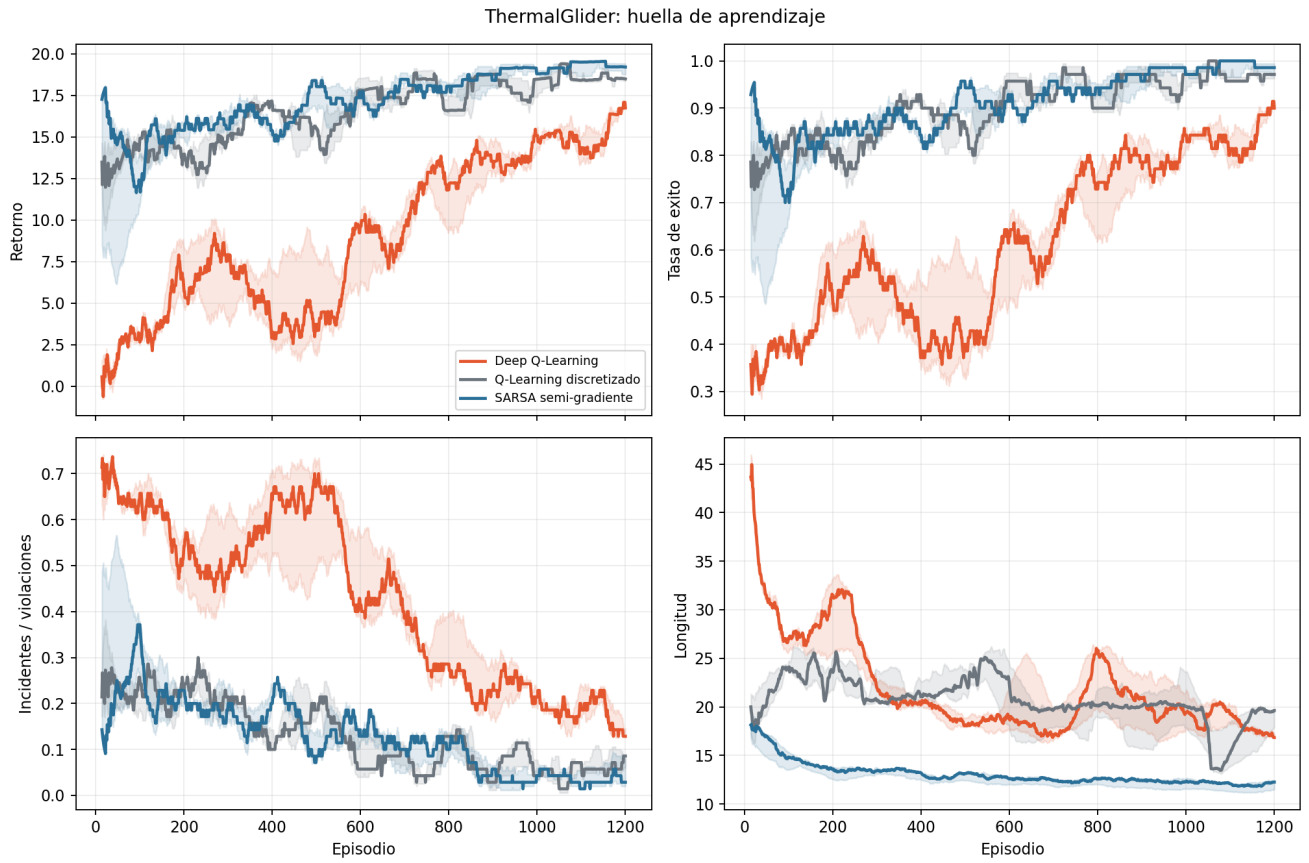


Figura 3: Huella de aprendizaje de los métodos con aproximación.

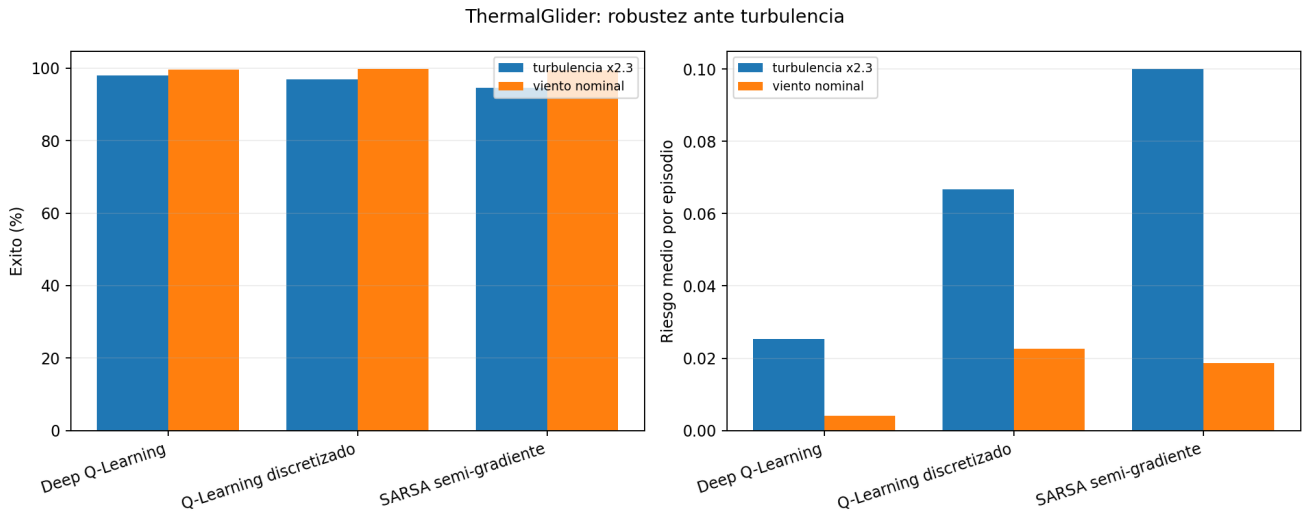


Figura 4: Robustez de los agentes ante aumento de turbulencia.

| Agente                  | Condición        | Éxito mínimo | Éxito máximo | Riesgo medio | Desv. riesgo |
|-------------------------|------------------|--------------|--------------|--------------|--------------|
| Deep Q-Learning         | Turbulencia x2.3 | 0.952        | 1            | 0.025        | 0.023        |
| Deep Q-Learning         | Viento nominal   | 0.988        | 1            | 0.004        | 0.007        |
| Q-Learning discretizado | Turbulencia x2.3 | 0.936        | 0.988        | 0.067        | 0.039        |
| Q-Learning discretizado | Viento nominal   | 0.992        | 1            | 0.023        | 0.032        |
| SARSA semi-gradiente    | Turbulencia x2.3 | 0.912        | 0.964        | 0.1          | 0.045        |
| SARSA semi-gradiente    | Viento nominal   | 0.988        | 1            | 0.019        | 0.018        |

Tabla 4: Variabilidad entre semillas en el estudio aproximado.

La evaluación por semillas refuerza la lectura anterior. Bajo turbulencia, DQN conserva un éxito mínimo de 0.952,

mientras que SARSA semi-gradiente baja a 0.912. La media de riesgo de DQN es 0.025 frente a 0.1 en SARSA semi-gradiente.

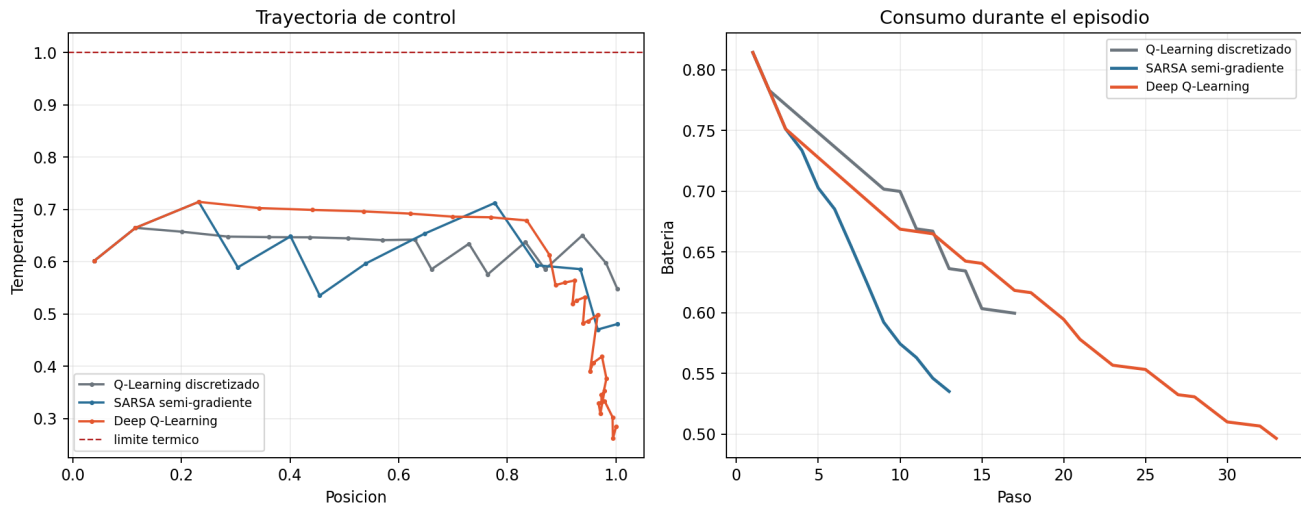


Figura 5: Trayectorias representativas en `ThermalGliderEnv`

Las trayectorias representativas muestran el compromiso entre rapidez y seguridad. SARSA semi-gradiente avanza de forma más agresiva y termina antes; DQN adopta una trayectoria más conservadora. La figura se usa como apoyo cualitativo, no como prueba aislada de superioridad.

## 7. Conclusiones

En el entorno discreto `RiverCourierEnv`, **Monte Carlo off-policy** y **SARSA** son las opciones más consistentes, pues alcanzan éxito completo y riesgo nulo en las dos condiciones evaluadas. Mientras, **Q-Learning** resuelve muchos episodios, pero presenta mayor variabilidad entre semillas y episodios más largos.

En el entorno continuo `ThermalGliderEnv` la aproximación de funciones es necesaria para tratar el estado sin una discretización excesivamente pobre. **SARSA semi-gradiente** consigue trayectorias más cortas en condiciones nominales, pero su robustez cae con turbulencia. En cambio, **DQN** tiene mayor coste algorítmico, aunque ofrece mejor compromiso cuando se exige seguridad bajo perturbaciones.

La comparación muestra que el retorno medio no basta para evaluar estos agentes. La longitud, el éxito, el riesgo y la variabilidad entre semillas son necesarios para distinguir políticas rápidas, seguras y robustas. Esta lectura es especialmente importante en entornos complejos, donde llegar antes no equivale siempre a aprender mejor.

## Reproducibilidad

Los notebooks principales de esta parte son:

- `01_gymnasium_y_agente.ipynb`: contrato Gymnasium y bucle común.
- `02_estudio_riesgo_tabular.ipynb`: Monte Carlo, SARSA y Q-Learning en `RiverCourierEnv`.
- `03_control_continuo_aproximado.ipynb`: SARSA semi-gradiente, DQN y referencia discretizada en `ThermalGliderEnv`.

Cada notebook localiza la raíz del proyecto y, si es necesario, intenta clonar el repositorio configurado en <https://github.com/NaoKB/EML>. Los resultados se reconstruyen mediante `src/artifacts.py` cuando faltan los CSV o las figuras.